VLM Reliability Toolkit

Detailed demo script for four presenters

Unified inference-time hallucination detection and attribute-binding correction for vision-language models

Prithviraj More · Kavyaansh Chandrikapure · Harsh Mulimani · Yash Mulimani · guide: Prof. Pallavi Dhade, Dept. of CSE (AIML), PCCOE Pune

Part	Presenter	What you explain	Paper	What you show live	Time
1	Yash	The problem, related work, problem formulation, contributions, how hallucination is measured	I, II, III	The problem figure; the comparison with prior methods	5 min
2	Prithvi	Architecture, candidate generation (Eq. 2), claim extraction	IV-A, IV-B, IV-C	Scene A: Gemini live on a street image; Claims page	6 min
3	Kavyansh	Verification (Eq. 3, 4), counts and relations, correction, reliability score (Eq. 5), tuning	IV-D, IV-E, IV-F, V-B, VI-D	Scene B: a wrong caption corrected; terminal swap example	7 min
4	Harsh	Decision policy and re-ranking, benchmarks, results, ablations, limitations, conclusion	IV-F, V, VI, VII, VIII	Scene C: re-ranking; Evaluation and Experiments pages	7 min

Total about 25 minutes of presentation plus 5–6 minutes of questions. Each part follows the same pattern: explain with a diagram, run one thing live, hand off. Shaded purple boxes are the words to say; grey boxes are the clicks; green “Point at” lines tell you where to direct the audience.

How to read this script

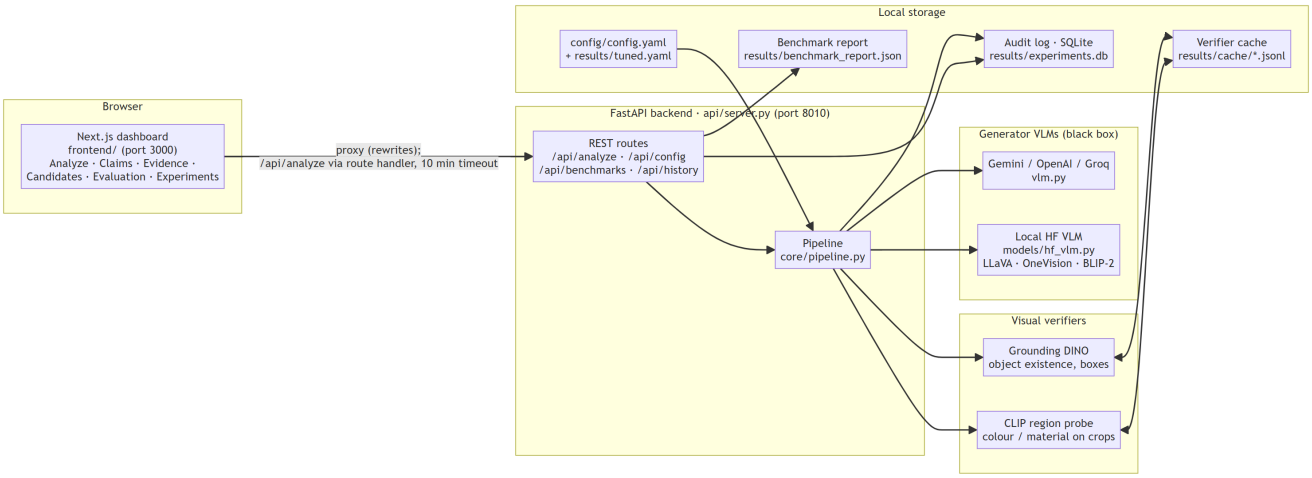
Box	Meaning
SAY (purple)	Words to speak. Read it a few times; say it in your own words on the day.
DO (grey)	Exactly what to click or type, in order.
POINT AT	The thing on screen the audience should look at while you talk.
HAND-OFF	Your last line; it introduces the next speaker.

The one-sentence pitch (everyone should be able to say this)

> **SAY:** We built a training-free layer that sits after any vision-language model, breaks its answer into small claims, checks each claim against the image with a detector and CLIP, scores how reliable each answer is, and then accepts, corrects, re-ranks or abstains — changing only what the image contradicts. On LLaVA-1.5-7B it cuts hallucinated captions from 63.8% to 57.2% while losing less than one point of object recall.

The system at a glance (shared reference for all four parts)

System overview — docs/images/system-overview.png



The browser talks to the Next.js dashboard, which proxies to the FastAPI backend on port 8010. The backend runs the pipeline: a generator VLM (API or local Hugging Face) produces candidates, and two visual verifiers (Grounding DINO and CLIP) check them. Verifier outputs are cached on disk by image hash; every analysis is written to a SQLite audit log; benchmark reports are served to the Evaluation page.

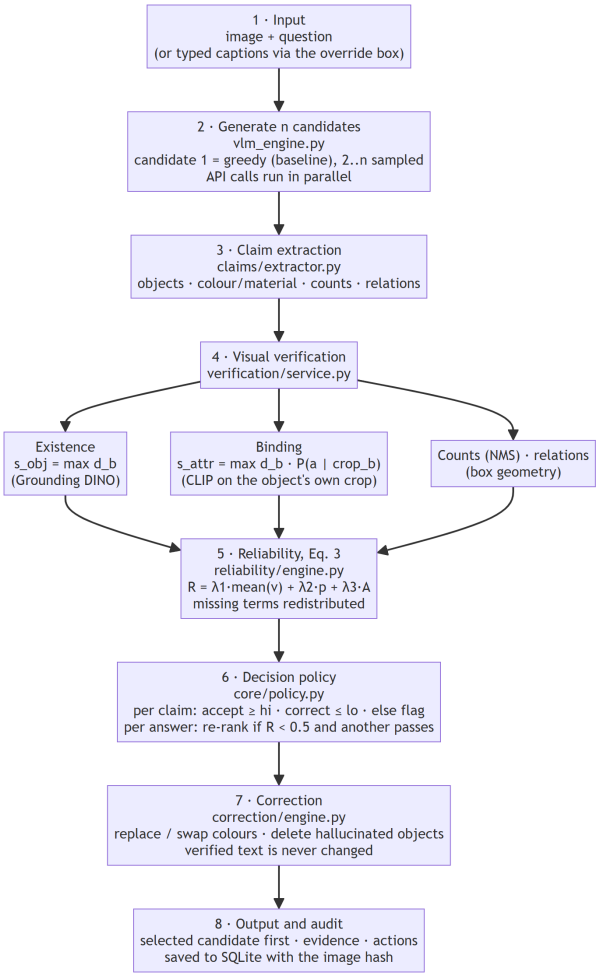
The seven components of Section IV-A, mapped to the code

Component (paper)	Module in the repository	Responsibility	Presenter
VLM execution engine	vlm.py, models/hf_vlm.py	Loads the base model: API providers or local LLaVA / OneVision / BLIP-2	Prithvi
Inference orchestrator	vlm_engine.py, core/pipeline.py	Requests n candidates, runs the stages in order, records timings	Prithvi
Diagnostic engine	claims/extractor.py, verification/	Extracts claims and verifies them against the image	Prithvi / Kavyansh
Reliability and scoring engine	reliability/engine.py	Agreement A_j and reliability R_j (Eq. 5)	Kavyansh
Correction and decision layer	core/policy.py, correction/engine.py	Accept / correct / abstain per claim; re-rank; span edits	Harsh / Kavyansh
Evaluation and benchmarking	evaluation/, kaggle/	POPE and CHAIR harness, ablations, tuning, reports	Harsh
Dashboard and reporting	frontend/, api/server.py, database/db.py	Analyze, Claims, Evidence, Candidates, Evaluation, Experiments	Prithvi / Harsh

Design principles to repeat throughout

- **Visual evidence decides, not another LLM.** Every decision comes from detector boxes and crop-level CLIP probabilities.
- **Bind attributes to the object's own region.** A blue sky cannot support “blue shirt”.
- **Never invent a missing signal.** Missing token confidence or agreement drops out of the score.
- **Correct only with positive evidence; otherwise abstain.** Uncertain claims are flagged, not guessed.
- **Minimal edits.** Corrections edit character spans; verified words are never rewritten.

One analysis request, step by step



1 · Input	Image and question from the Analyze page, or typed captions through the override box.
2 · Generate	n = 3 candidates. Candidate 1 is greedy (the baseline); 2 and 3 are sampled. API calls run in parallel.
3 · Extract	Rule-based claims: objects, colour/material, counts, relations. Each keeps its text span.
4 · Verify	Existence with Grounding DINO (Eq. 3); binding with CLIP on the object's crop (Eq. 4); counts by NMS; relations by geometry.
5 · Score	$R = 0.6 \cdot \text{mean } v + 0.2 \cdot p + 0.2 \cdot A$ over the available terms (Eq. 5).
6 · Decide	Per claim: accept / correct / abstain. Per answer: re-rank if $R_1 < 0.5$ and another candidate passes.
7 · Correct	Delete, replace, swap or recount — only with positive evidence.
8 · Output	Selected answer, evidence and actions; saved to SQLite with the image hash.

Source: docs/images/analysis-pipeline.png. Stages 1–3: Prithvi · stages 4, 5 and 7: Kavyansh · stage 6 and the benchmark path: Harsh. Yash sets up the problem before all of this.

The problem, related work, and what we contribute

Paper sections: I Introduction, II Related Work, III Problem Formulation

1.1 Open with the problem (60 s)

SAY — Yash: Good morning. Vision-language models like LLaVA, BLIP-2 and Gemini can look at a picture and write a fluent description. But they make two kinds of mistakes that matter. First, object hallucination: the model mentions something that is simply not in the image — a surfboard that isn't there. Second, attribute-binding errors: the objects are real and the colours are real, but they are attached to the wrong objects. The picture shows a red cup on a blue table, and the model says a blue cup on a red table. These two failures need different checks, because confirming that a cup exists tells you nothing about whether its colour is right.

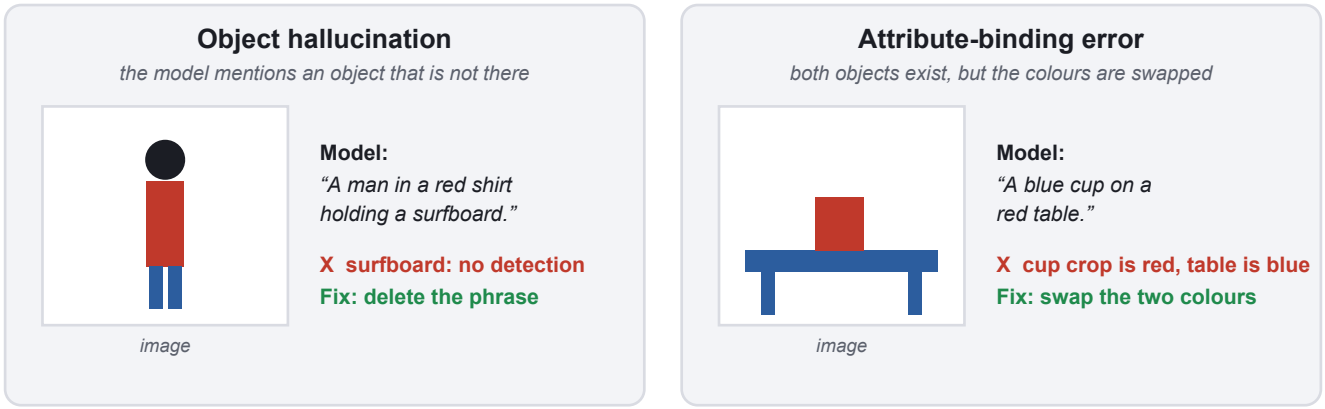


Figure 1. The two failure modes and the repair each one needs.

1.2 What exists already, and the gap (45 s)

SAY — Yash: People have attacked this before. LURE trains a separate model to rewrite hallucinated descriptions. VOLCANO has the model critique and revise itself. OPERA changes the decoding so the model trusts its own text less. Woodpecker, the closest to us, corrects answers after generation using visual models — but it still relies on language models for some steps. What was missing was a transparent, claim-level policy that checks both existence and attribute binding, edits only what the evidence contradicts, and is willing to say "I'm not sure" instead of guessing. That is our question: can we detect and selectively correct both error types in one training-free, inference-time framework, while keeping everything that was correct?

Method	How it works	Training?	Checks binding?	Can abstain?
LURE	Trained revisor rewrites the description	Yes	No	No
VOLCANO	Self-feedback revision by the same model	Yes	No	No
OPERA	Decoding-time penalty on over-trust	No	No	No
Woodpecker	Post-hoc correction with visual experts + LLM stages	No	Partly	No
Ours	**Deterministic claims → detector + region CLIP → evidence-gated edits**	**No**	**Yes**	**Yes**

1.3 Problem formulation (45 s) — paper Section III

SAY — Yash: Formally: we take the model's answer y and split it into atomic claims. A claim is one of four kinds — an object exists, an object has a colour or material, an object has a count, or two objects are in a spatial relation. A false existence claim is a hallucination. A false attribute claim is a binding error when the claimed value actually belongs to a different object in the same picture — that is Equation 1 in the paper. Otherwise it is a plain attribute error. The distinction matters because the repairs differ: delete a hallucinated object, swap a mis-bound attribute, and replace a plain attribute error with what we actually observe. And the goal is not to delete everything doubtful — correct content must survive.

Claim type	Form	Example	Repair if false
Object (existence)	<o>	“a dog” → dog exists	delete the mention
Attribute (binding)	<o, a>, a in 11 colours or 8 materials	“a red shirt” → shirt is red	swap (binding error) or replace (plain error)
Quantity	<o, m>	“two dogs” → count(dog) = 2	recount to the detected number
Relation	<o_1, r, o_2>, r in {left of, right of, above, below, next to, near}	“a cat near a car”	abstain (no safe substitute)

1.4 What we contribute (45 s)

SAY — Yash: So we built the VLM Reliability Toolkit. It makes four contributions. One: claim-level verification that separates object hallucination from attribute-binding errors, and checks every attribute on its own object's detected region. Two: a reliability score that combines visual evidence, token confidence and agreement between candidates — and never invents a signal that isn't available. Three: evidence-gated decisions: we correct only when the image gives positive evidence, abstain otherwise, and re-rank across several candidates. Four: a reproducible evaluation on POPE and CHAIR with five ablations, including an honest look at where the method does not help. The headline: on LLaVA-1.5-7B, hallucinated captions drop from 63.8 to 57.2 percent with less than a point of recall lost.

Contribution	Where in the paper	Presented by
Claim-level verification with region-bound attributes	Section IV-D	Kavyansh
Reliability score with unavailable terms redistributed	Section IV-E	Kavyansh
Evidence-gated accept / correct / abstain policy + re-ranking	Section IV-F	Harsh
Reproducible two-stage evaluation, five ablations	Sections V and VI	Harsh

1.5 How hallucination is measured (30 s) — Section II

SAY — Yash: A word on how this is measured, because it shapes our results. CHAIR compares the objects mentioned in a caption with the ground-truth annotations. POPE turns object presence into yes-or-no questions. HallusionBench diagnoses language bias and visual illusions. And for attribute binding and composition there are ARO, Winoground and SugarCrepe. We report POPE and CHAIR here; the compositional benchmarks are our next step.

Benchmark	What it tests	Used here?
CHAIR [7]	Objects mentioned in captions vs ground truth	Yes — 500 COCO images

Benchmark	What it tests	Used here?
POPE [8]	Object presence as balanced yes/no polling	Yes — 3 splits × 2,700 questions
HallusionBench [16]	Language bias and visual illusion	No
ARO [9], Winoground [10], SugarCrepe [11]	Attributes, relations, order; compositional matching	Future work

Questions you own

If asked…	Answer
How is this different from Woodpecker?	Woodpecker also corrects post hoc, but uses language models in some stages. Ours is deterministic end to end, represents attribute binding explicitly, and can abstain.
Why do the two failure modes need different checks?	Detecting a cup says nothing about its colour. Existence is checked with the detector; binding is checked with CLIP inside that object's own box.
Why not test on Winoground or SugarCrepe?	They are the right benchmarks for binding, and they are planned. POPE and CHAIR only measure object hallucination, so the paper says binding correction is not yet quantitatively evaluated.

HAND-OFF → *That is the problem. Prithvi will show how the system is built, and how an answer becomes claims.*

The architecture, candidate generation and claim extraction

Paper sections: IV-A System Architecture, IV-B Candidate Generation, IV-C Claim Extraction

2.1 The architecture (60 s) — show the system overview page

DO

1. Switch to the PDF/slide with **docs/images/system-overview.png** (page 2 of this script), or open docs/ARCHITECTURE.md in the browser.

SAY — Prithvi: Here is the system. It is a layer that runs after any VLM — we never train or fine-tune anything. It treats the model as a black box, or as a white box when token probabilities are available. There are seven components: the VLM engine; an orchestrator that asks for n candidate answers; a diagnostic engine that extracts and verifies claims; the reliability engine; the decision and correction layer; the benchmark module; and the dashboard you're looking at. And there is one rule that runs through all of it: no language model judges another language model. Every decision is based on detector boxes and image crops, so every decision can be shown and audited.

POINT AT: the two verifier boxes (Grounding DINO, CLIP) and the cache between them and the pipeline.

2.2 Candidate generation (45 s) — Section IV-B, Eq. 2

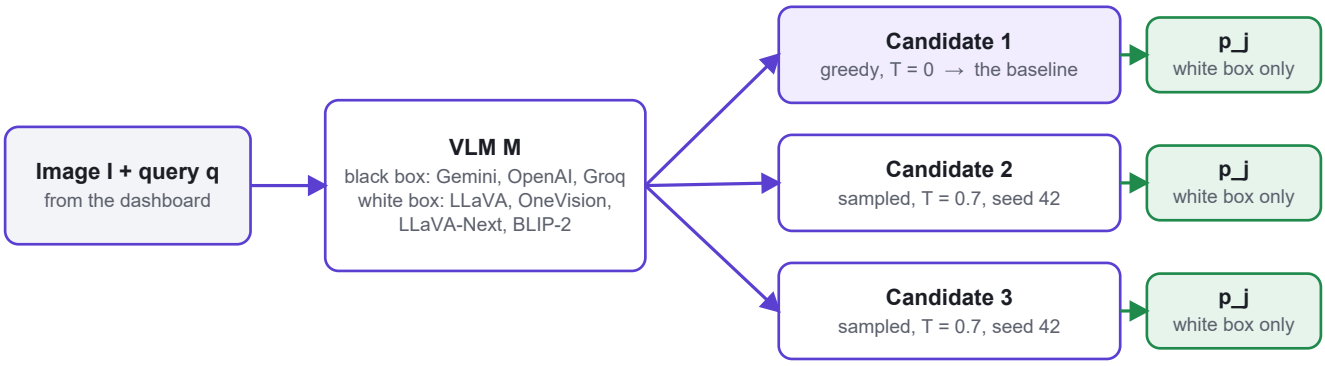


Figure 2. Candidate generation. Candidate 1 is always greedy and is the uncorrected baseline.

SAY — Prithvi: For every image and question we ask for three candidates. Candidate 1 uses greedy decoding — that is our baseline, so every number we report compares the toolkit against the same model's normal answer. Candidates 2 and 3 are sampled at temperature 0.7 with seed 42, at most 128 new tokens. For a local white-box model we also compute the token confidence p_j: the exponential of the average log-probability of the generated tokens. That is Equation 2. An API model like Gemini does not expose this, so we mark it unavailable — we never estimate it. And if a generation fails, its slot is kept and marked GENERATION_FAILED, so it can never silently replace the baseline.

Eq. 2	$p_j = \exp[\ (1/T_j)\ \Sigma_t \log P(w_t \mid w_{<t}, I, q)\ ]$ — length-normalised sequence probability over the T_j generated tokens

2.3 Claim extraction (45 s) — Section IV-C

Candidate 1 (Scene C):

“A golden retriever dog and a white cat sit on the grass near a green car and a red bicycle.”

rule-based: COCO nouns, colours, relations

dog exists object claim	**cat exists** object claim	**car exists** object claim	**bicycle exists** object claim
cat is white attribute claim	**car is green** attribute claim	**bicycle is red** attribute claim	**cat near car** relation claim

Each claim keeps its character span in the sentence, so correction can later edit exactly those words.

Figure 3. A real candidate from Scene C and the eight claims extracted from it.

SAY — Prithvi: Next, each candidate is split into atomic claims. This is deliberately not done by a language model — it is a deterministic, rule-based extractor, so the same sentence always gives the same claims. It uses the 80 COCO object classes with the CHAIR synonym list — “man” and “woman” map to person — plus common clothing and scene words, 11 colours and 8 materials. An attribute binds to the noun it modifies, and for a relation the target is the nearest object after the relation phrase. Here, one sentence becomes eight claims: four objects, three colours and one relation. Each claim remembers exactly where it is in the sentence, which is what lets us edit later without rewriting.

2.4 Live demo — Scene A: a real model, live (90 s)



Scene A input: test_images/Gemini_Generated_Image_8jkxpp8jkxpp8jkx.png

DO

1. Go to **http://127.0.0.1:3000/analyze**. Make sure the **Controlled candidate override** box is empty.
2. Upload **Gemini_Generated_Image_8jkx…png**. Question: *Describe the objects and their colors.*
3. Provider **Gemini** (gemini-2.5-flash), candidates **3**. Click **Run reliability analysis**.
4. While it runs (10–20 s), say the box below. When it returns, open the **Claims** page and the claim graph.

SAY — Prithvi: This is a live call to Gemini 2.5 Flash. It's writing three descriptions of this street in parallel. When they come back, each one goes through exactly the pipeline I just described. On the Claims page you can see the claims it found and how they connect: objects, their instances, and their colours. My teammates will now show what happens to each of these claims.

Claim (from the rehearsal run)	Status	v	Action
car exists	SUPPORTED	0.73	ACCEPT
car is yellow (taxis)	SUPPORTED	0.67	ACCEPT
person exists	SUPPORTED	0.80	ACCEPT
tree exists · building exists	SUPPORTED	0.58 · 0.40	ACCEPT
car is blue (“blue sedans”)	CONTRADICTED	0.15	ABSTAIN — flagged, left unchanged
tree is green	SUPPORTED	0.19	ABSTAIN — flagged, left unchanged

Gemini's wording changes from run to run, so the exact claims may differ. Expected overall: most claims accepted, one or two colour claims flagged, decision ABSTAIN with R around 0.55.

- **Fallback** (no Wi-Fi or a Gemini error): open the **Experiments** page and reopen this morning's 8jkx run. Say: “every run is stored in the audit log — here's the one we ran this morning.”

Questions you own

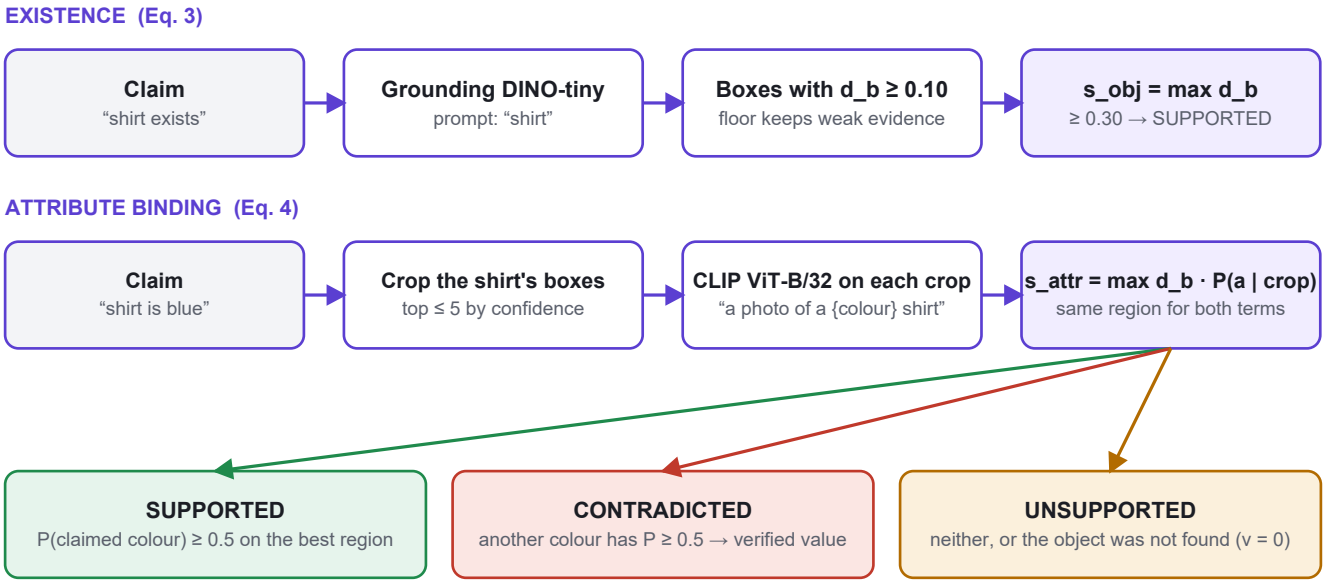
If asked…	Answer
Why not use an LLM to extract claims?	Determinism and auditability: the same sentence always gives the same claims, and no model is judging another model. The cost is a closed vocabulary (Section VII).
Does it work with any VLM?	Yes. API models are black boxes; local models are white boxes that also give p_j. Both share one engine interface.

HAND-OFF → *We now have claims. Kavyansh will show how we check each one against the image, and how an answer is scored.*

Visual verification, correction, and the reliability score

Paper sections: IV-D Visual Verification, IV-E Reliability Scoring, IV-F Correction, V-B Tuning, VI-D Example

3.1 How a claim is checked (90 s) — Section IV-D



Only the object's own crop is probed, so colour elsewhere in the image can never support the claim.
AMBIGUOUS = the verifier could not run; such claims are never scored or corrected.

Figure 4. Existence and attribute-binding verification, and the three outcomes.

SAY — Kavyansh: Every claim gets a score v_i between 0 and 1, and a status. For an existence claim we give the object word to Grounding DINO, an open-vocabulary detector, and take the confidence of its best box. That is Equation 3. We keep boxes down to 0.10, so weak evidence still shows up as a real number instead of zero, and the claim is supported at 0.30 or above. For an attribute claim we crop each of the object's top five boxes and ask CLIP to compare prompts like "a photo of a red shirt", "a photo of a blue shirt", across all eleven colours. The binding score multiplies detection confidence by the crop probability, on the same region — Equation 4. If the claimed colour reaches 0.5, it's supported. If a different colour reaches 0.5, it's contradicted, and we record that colour as the verified value. The crucial point: we only look inside the object's own box, so colour elsewhere in the picture can't support the claim.

Equation	Definition	Threshold / rule
Eq. 3 existence	$s_{obj}(o)$ = max over boxes b in B(o) of d_b, with $d_b \geq 0.10$	SUPPORTED if $s_{obj} \geq 0.30$, else UNSUPPORTED (OBJECT_HALLUCINATION)
Eq. 4 binding	$s_{attr}(o, a)$ = max over b of $d_b \cdot P(a \mid crop_b)$, top ≤ 5 boxes	SUPPORTED if $P(a) \geq 0.5$; CONTRADICTED if another a′ has $P \geq 0.5$ (ATTRIBUTE_BINDING_ERROR)

3.2 Counts and relations (30 s)

> **SAY — Kavyansh:** Counts and relations reuse the same boxes. For a count, we remove duplicate boxes with non-maximum suppression at IoU 0.5 and compare the number left with the number claimed. For relations we use box geometry: centres for left, right, above and below, and a distance rule for near and next to. If either object can't be found, the relation is ambiguous and we never auto-correct it.

Claim	Rule in verification/service.py	Score v_i
count(o) = m	boxes with $d_b \geq 0.30$, greedy NMS at IoU 0.5; compare survivors with m	s_{obj} if equal, else 0 (observed count recorded)
left of / right of	compare box-centre x	$\min(s_{obj}(o_1), s_{obj}(o_2))$ if it holds, else 0
above / below	compare box-centre y	same
near / next to	L1 centre distance < 2 × the larger box's (width + height)	same; AMBIGUOUS if an object is not found

Status	Meaning	Used in the score?	Can be corrected?
SUPPORTED	Evidence for the claim	Yes	—
CONTRADICTED	Evidence for a different value	Yes	Yes, with the verified value
UNSUPPORTED	No evidence (e.g. no detection)	Yes	Objects: delete; others: abstain
AMBIGUOUS	Verifier could not run	No (excluded)	Never

3.3 Live demo — Scene B: a wrong caption, corrected (2 min)

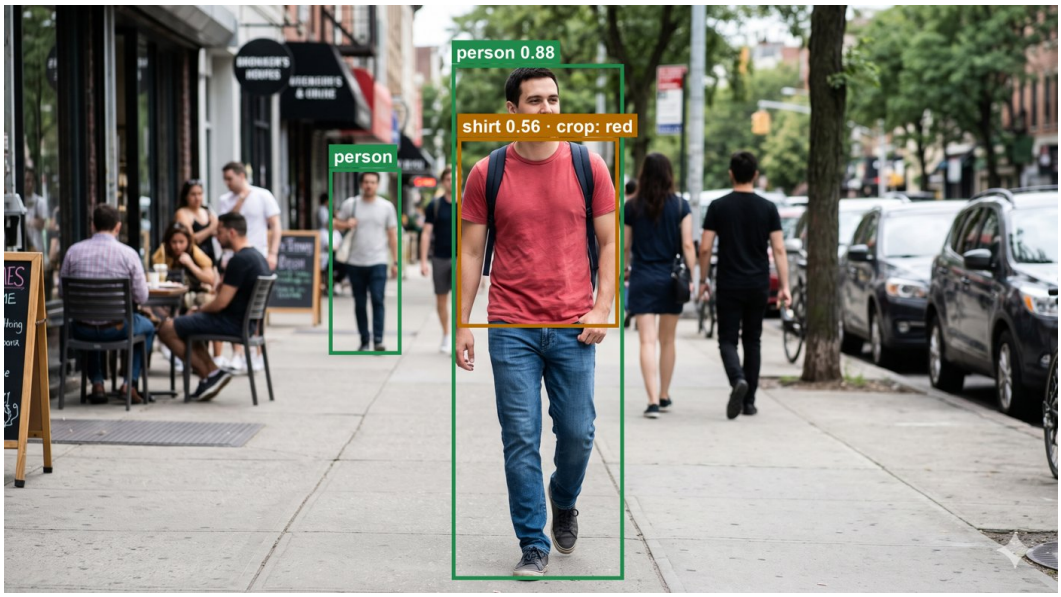


Figure 5. Scene B with the real Grounding DINO boxes from the rehearsal run (scores are s_{obj}). There is no surfboard box anywhere.

DO

1. Upload **Gemini_Generated_Image_7nkn…png**. Question: *Describe the person.*
2. Open **Controlled candidate override** and paste exactly: *A man in a blue shirt walks down the street holding a surfboard.*
3. Click **Run reliability analysis** (1–5 s from the cache).
4. Show, in order: the **Evidence canvas**, the **claims table**, the shirt **crop**, and the **Correction diff**.

SAY — Kavyansh: We typed a deliberately wrong caption: the man's shirt is red, and there's no surfboard. On the evidence canvas, the detector finds the person at 0.88 and the shirt at 0.56. It finds no surfboard at all — its best score is 0.12, below the 0.30 line.

POINT AT: the shirt box, then the claims table row by row.

Claim	Evidence	v_i	Status	Action
person exists	best box 0.88	0.88	SUPPORTED	ACCEPT
shirt exists	best box 0.56	0.56	SUPPORTED	ACCEPT
shirt is blue	shirt crop: red 0.59, blue 0.05	0.04	CONTRADICTED → red	CORRECT: replace blue → red
surfboard exists	no box ≥ 0.30 (best 0.12)	0.12	UNSUPPORTED	CORRECT: delete the clause

SAY — Kavyansh: Inside the shirt's own crop, CLIP gives red 0.59 and blue only 0.05. Red is above 0.5, so the claim is contradicted and red becomes the verified value. The surfboard has no detection, so the phrase “holding a surfboard” is removed. And look at the result: “A man in a red shirt walks down the street.” Every word that was verified is exactly as we typed it.

Before	A man in a ~~blue~~ **red** shirt walks down the street ~~holding a surfboard~~.
After	**A man in a red shirt walks down the street.**

3.4 The four correction operations (60 s) — Section IV-F

SAY — Kavyansh: Correction never regenerates the sentence. It edits the character spans that the extractor recorded, and there are only four operations. And every one needs positive evidence for the replacement — if there isn't any, the claim is flagged and left alone.

Operation	When	Example
Replace	Attribute contradicted; the crop clearly shows another value	“blue shirt” → “red shirt” (Scene B)
Swap	Two attributes in a sentence, each contradicted by the other's claimed value (a binding error, Eq. 1)	“blue shirt and red jeans” → “red shirt and blue jeans”
Recount	Count claim; NMS gives a different number	“three dogs” → “two dogs”
Delete	Object with no detection above the floor ($s_{obj} < 0.30$)	“a dog and a cat” → “a dog”; “…holding a surfboard” removed

3.5 The swap, live in the terminal (45 s) — Section VI-D

DO

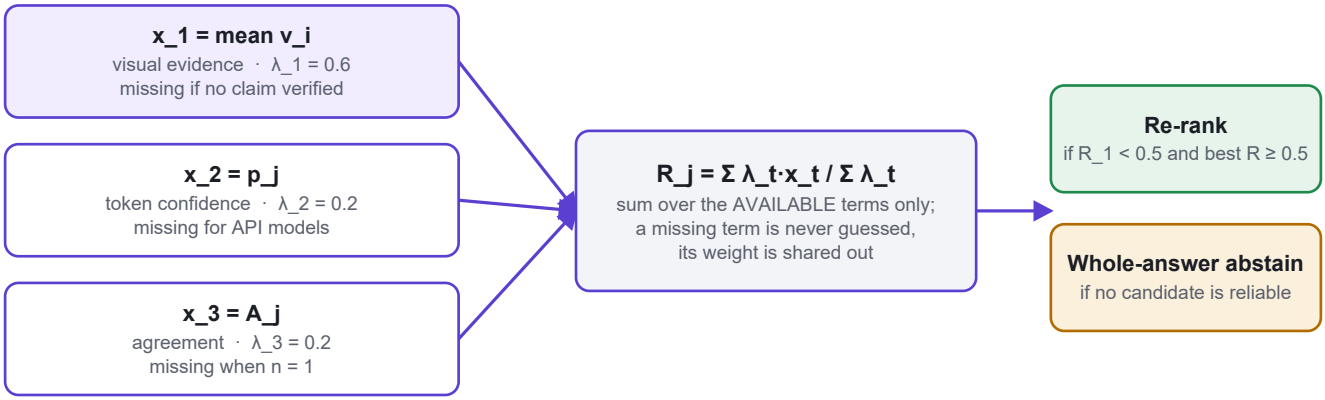
1. In a terminal in the project folder: .venv\Scripts\python demo_paper_example.py (no models or API needed; runs in about a second).

Claim	Verification (Table VI of the paper)	Action printed
person exists	pass	ACCEPT
shirt is blue	fail: region is red	CORRECT swap: shirt ↔ jeans
jeans are red	fail: region is blue	CORRECT swap: jeans ↔ shirt
surfboard exists	fail: not detected	CORRECT delete
beach exists	pass	ACCEPT

Printed: Original: *A person in a blue shirt and red jeans stands on the beach holding a surfboard.* → Corrected: *A person in a red shirt and blue jeans stands on the beach.* Decision CORRECT.

SAY — Kavyansh: This is the example from Section VI-D of our paper. Both colours are real, but they're on the wrong garments — a classic binding error. Each region shows the other's claimed colour, so the policy swaps them rather than replacing both independently, and deletes the surfboard. This example uses fixed verifier outputs to demonstrate the policy; the paper is clear that it's an illustration, not a measured result.

3.6 One score per answer (75 s) — Eq. 5



R is used only at the answer level. Each claim's own action uses its own v_i (next page).

Figure 6. The reliability score combines three signals and never invents a missing one.

SAY — Kavyansh: Every candidate answer gets one reliability score, R, between 0 and 1. It combines three signals. The main one, with weight 0.6, is the visual evidence: the mean verification score of its claims. Then, with 0.2 each, the model's own token confidence, and agreement — the fraction of its claims that at least one other candidate also makes. The important design choice is what happens when a signal is missing. Gemini gives no token probabilities; a single typed caption has nobody to agree with. We never invent a value. We just sum over the terms we have and divide by their weights, so the remaining weights are scaled up proportionally.

Available terms	Denominator	Formula	Live example
v, p, A (local model, n = 3)	1.0	R = 0.6v + 0.2p + 0.2A	—
v, A (API model or typed captions, n = 3)	0.8	R = (0.6v + 0.2A) / 0.8	Scene C: 0.865
v only (one typed caption)	0.6	R = v (the mean claim score)	Scene B: 0.398

Scene B check: mean of 0.879, 0.555, 0.116 and 0.041 = 0.398 — exactly the R the dashboard shows.

3.7 Where the thresholds and weights come from (60 s) — Section V-B

SAY — Kavyansh: The thresholds Harsh is about to use are not hand-picked. We tuned them once, on 50 POPE images that are never used in any reported number. The low threshold is the value on a grid from 0.10 to 0.45 that gives the best F1; the high threshold is the smallest value above it at which accepted claims are 95% precise. That gave 0.15 and 0.20 for objects, with an F1 of 0.848 and under 1% abstention. Attributes keep 0.15 and 0.40, because POPE has no attribute questions to tune on.

SAY — Kavyansh: We also searched the weights to minimise calibration error. On POPE that search puts all the weight on token confidence — because POPE answers are a single yes-or-no token, so token confidence is trivially well calibrated there. But applied to captions, that score ignores the image, and CHAIR gets worse: 59.4 instead of 57.2. So we keep the visual-dominant weights 0.6, 0.2, 0.2. The search result is still recorded, and tuning the weights is opt-in, with the visual weight kept at 0.5 or more.

Setting	Value	How it was chosen
Object / quantity / relation τ_{lo}, τ_{hi}	0.15, 0.20	Tuned on the 50-image held-out POPE split (F1 0.848, abstention 0.67%)
Attribute τ_{lo}, τ_{hi}	0.15, 0.40	Configured; POPE has no attribute claims
Grounding threshold / floor	0.30 / 0.10	Configured for Grounding DINO-tiny
λ_1, λ_2, λ_3	0.6, 0.2, 0.2	Kept; the ECE-optimal $\lambda_1 = 0$ hurts CHAIR (59.4 vs 57.2)
Re-rank threshold τ_R	0.5	Configured

Questions you own

If asked…	Answer
What if the detector misses a small object?	Then a real object can be deleted. That's why we report object recall (it drops only 0.8 points on CHAIR) and why the paper says a low detection score is not by itself proof of absence.
Why did it not correct the blue sedans in Scene A?	Its score (≈ 0.15) sat at the low threshold, inside the uncertain band, and the street has cars of many colours. Without clear evidence for one substitute, we flag instead of editing.
Are CLIP probabilities calibrated?	No — they express relative support among the configured labels. That's why we use them with thresholds and abstain in the middle.
Why 0.6 / 0.2 / 0.2?	Visual evidence should dominate. The ECE-optimal weights on POPE ignore the image and make CHAIR worse (59.4 vs 57.2).
Is R a probability of being correct?	No. Its ECE is about 0.21–0.27 on POPE. It ranks candidates well, but it needs human-annotated calibration before it can be used as a confidence.

HAND-OFF → *Every claim now has a score, and every answer a reliability. Harsh will show how we decide what to do with them — and whether it holds up over thousands of images.*

The decision policy, re-ranking, and results at scale

Paper sections: IV-F Decision Policy, V Setup, VI Results, VII Limitations, VIII Conclusion

4.1 The per-claim policy (60 s) — Table II

SAY — Harsh: Kavyansh gave every claim a score. Each claim is judged on its own score, with type-specific thresholds. At or above the high threshold we accept it. At or below the low threshold we correct it — but only if there is evidence for the fix: a missing object gets deleted, a contradicted colour gets the observed colour. Anything in between is flagged as uncertain and left exactly as written. And if the verifier couldn't run at all, we abstain — never a silent accept.

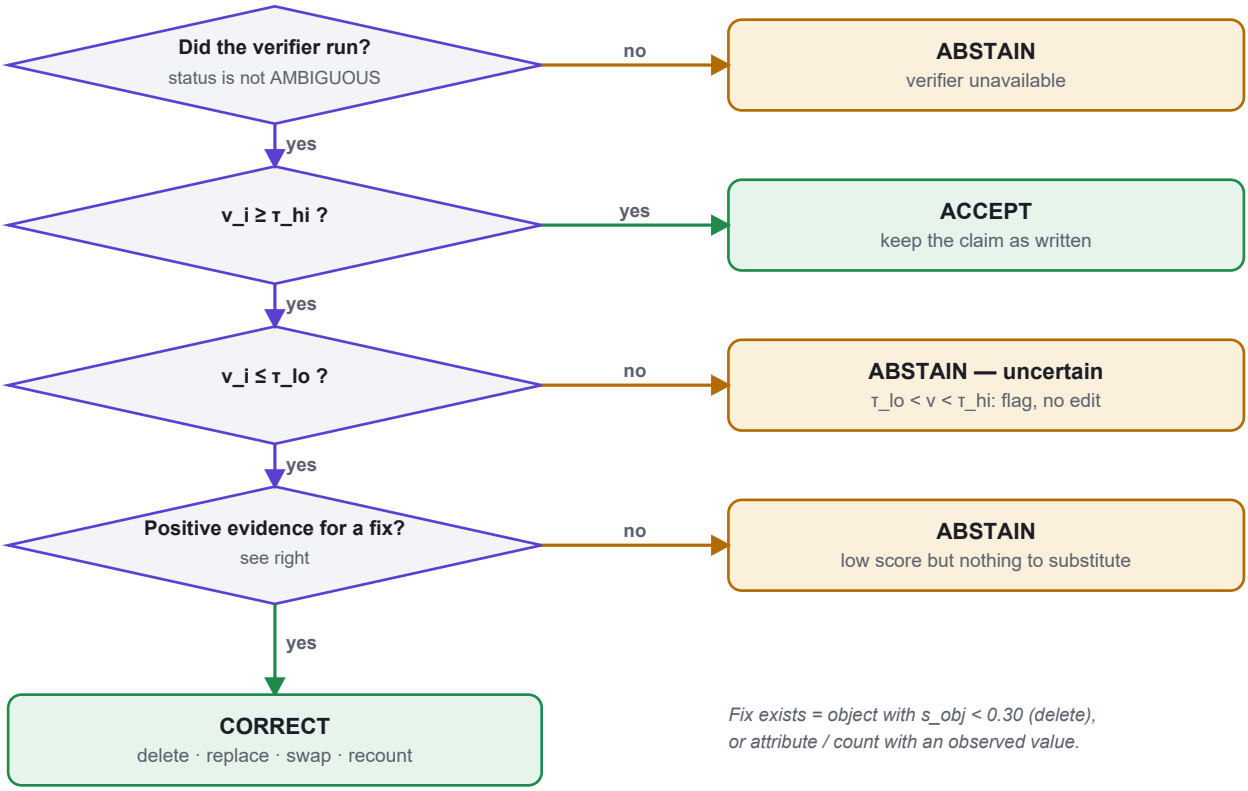


Figure 7. The per-claim decision. Thresholds are per type: objects τ_{lo} = 0.15, τ_{hi} = 0.20 (tuned); attributes 0.15 / 0.40.

4.2 The answer-level rule and re-ranking (45 s)

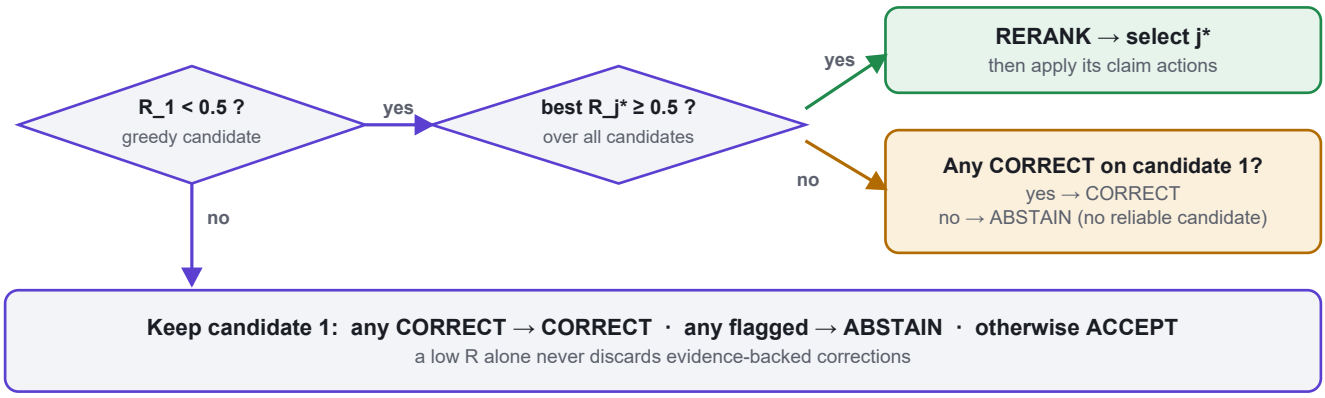


Figure 8. Answer-level decision with τ_R = 0.5.

SAY — Harsh: Then we decide for the whole answer. If the greedy answer's R is below 0.5 and some other candidate reaches 0.5, we switch to the best one: that's a re-rank. If nothing is reliable and there's no evidence-backed correction, the whole answer abstains. Notice Kavyansh's scene: R was only 0.40, yet the answer was corrected, not rejected — because a low R alone never throws away corrections that the image supports.

4.3 Live demo — Scene C: re-ranking three candidates (2 min)

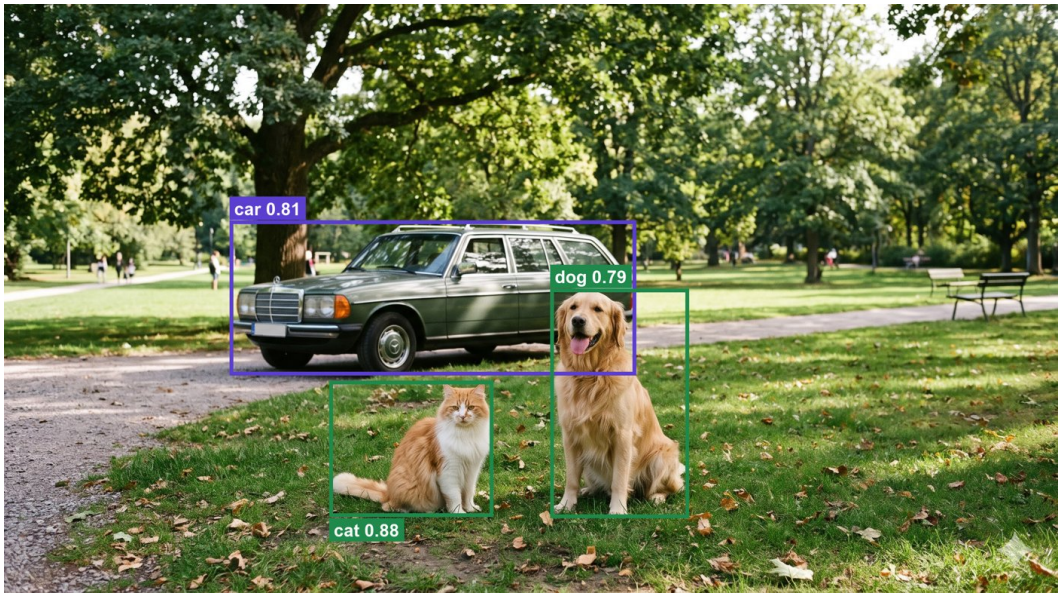


Figure 9. Scene C with the real detector boxes. There is no bicycle, bench-side cat pair or umbrella near the animals.

DO

1. Upload **Gemini_Generated_Image_dqp7…png**. Question: *Describe the objects and their colors.*
2. Paste into the override, three candidates separated by **|||**:
   A golden retriever dog and a white cat sit on the grass near a green car and a red bicycle. ||| *A dog and a cat sit near a car.* ||| *A brown dog sits on a bench next to two cats and a blue umbrella.*
3. Run. Show the **Decision panel** (RERANK), then the **Candidates** page (chart + three cards), then the **Reliability panel**.

	Candidate 1 (greedy)	**Candidate 2**	**Candidate 3**
Text	golden retriever dog, white cat, green car, red bicycle	A dog and a cat sit near a car.	brown dog on a bench, two cats, blue umbrella
Claims (v_i)	dog .79 · cat .88 · car .81 · bicycle 0 · white .18 · green .39 · red 0 · near .81	dog .79 · cat .88 · car .81 · near .81	dog .79 · bench .68 · cat .88 · umbrella .18 · brown .18 · blue 0 · two cats .88 · next to .68
mean v	0.481	0.8225	0.533
Agreement A	0.50 (4 of 8 shared)	1.00 (all 4 shared)	0.25 (2 of 8 shared)
R = (0.6v + 0.2A)/0.8	**0.486**	**0.865**	**0.462**
Outcome	R < 0.5 → not kept	**SELECTED (RERANK)**	below 0.5

> **SAY — Harsh:** The first answer is the detailed one, but it invents a bicycle and gets two colours unverifiable, so its mean evidence is only 0.48 and only half its claims are shared: R is 0.49, just under the line. The second answer is plain, but everything in it is verified and the other answers agree with all of it. Its four claims average 0.8225, agreement is 1, there's no token probability, so R is 0.6 times 0.8225 plus 0.2, all over 0.8 — 0.867. That is literally the worked example in Section IV-E of our paper, happening live. So the toolkit picks candidate 2.

POINT AT: the Decision panel text “Top candidate R=0.49 < 0.50; candidate-2 has R=0.86”, then the three cards (87% / 49% / 46%).

4.4 The setup (45 s) — Section V-A

SAY — Harsh: Now, does it hold up at scale? Everything that follows uses LLaVA-1.5-7B as the generator, Grounding DINO-tiny and CLIP ViT-B/32 as verifiers, and seed 42, run on Kaggle with two T4 GPUs — POPE on one, CHAIR on the other. Two benchmarks, because they measure different things.

	POPE	CHAIR
Measures	Object existence as yes/no questions	Hallucinated objects in free captions
Data	Official POPE questions, 450 COCO val2014 images × 3 splits (random, popular, adversarial) = 2,700 questions per split	500 COCO val2014 images, disjoint from tuning; ground truth from COCO instances + CHAIR synonyms
Query	“{question} Answer the question using a single word or phrase.” 8 tokens, 1 greedy candidate	“Describe this image in detail.” 128 tokens, 3 candidates
How the toolkit acts	A corrected existence claim turns “yes” into “no”	Delete / replace / re-rank in the caption
Metrics	Accuracy, precision, recall, F1, yes-ratio, abstention	$CHAIR_S$ (captions with ≥ 1 hallucination), $CHAIR_I$ (hallucinated mentions / all mentions), recall, words
Held out	50 further images (300 questions) for tuning only	—

4.5 A reproducible two-stage pipeline (45 s) — Section V-D

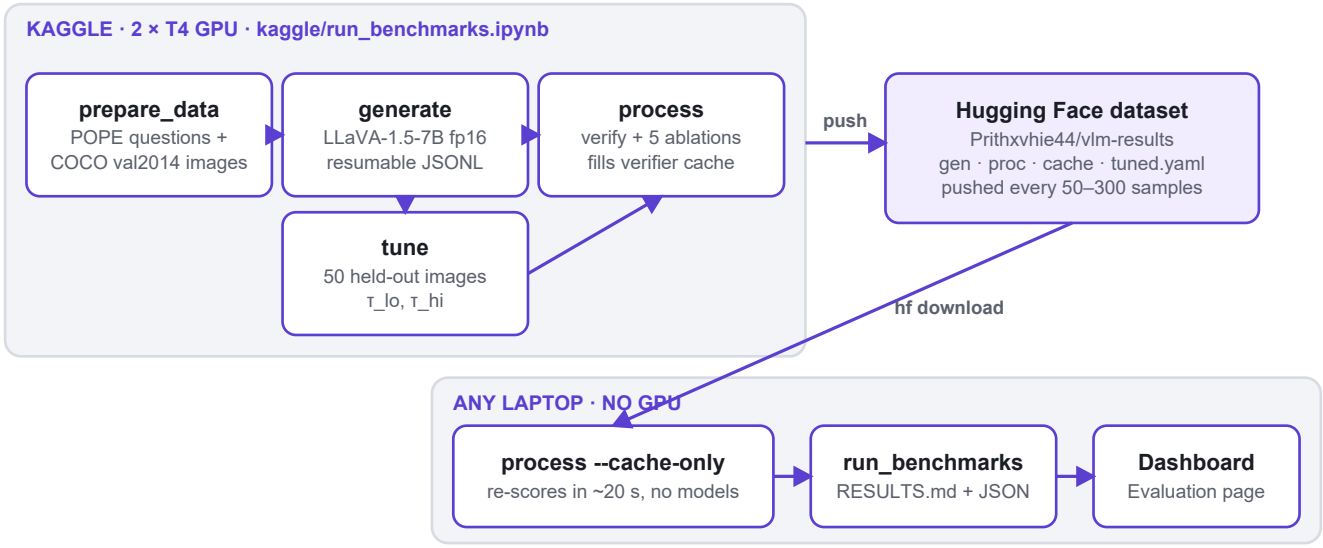


Figure 10. Benchmark path: GPU generation on Kaggle, results shared through Hugging Face, cheap CPU re-scoring anywhere.

SAY — Harsh: Generation on the GPU is the slow part, so it's separated from processing. Every Grounding DINO and CLIP output is cached, keyed by the image's SHA-256 hash and the query, together with how long it took. That means thresholds, weights and ablations can be re-evaluated on a laptop with no GPU: the full POPE — 16,200 rows — and CHAIR — 2,500 rows — each re-score in about 20 seconds. And because we store the compute time, latency is reported the same whether a row was computed or read from the cache. Results are pushed to Hugging Face every 50 to 300 samples, so a stopped Kaggle session simply resumes.

4.6 CHAIR results (75 s) — Table V, Fig. 2

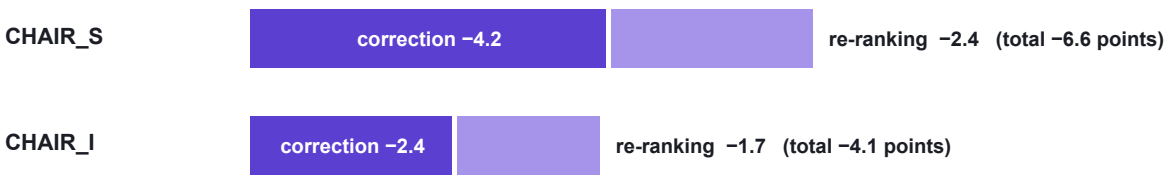
DO 1. Open **http://127.0.0.1:3000/evaluation** and scroll to the CHAIR table.

Variant (N = 500 images)	$CHAIR_S$ ↓	$CHAIR_I$ ↓	Recall ↑	Words	Latency
Base VLM (candidate 1, greedy)	63.8	29.6	76.2	86.4	1×
Full toolkit	**57.2**	**25.5**	75.4	83.9	2.67×
n = 1 (single candidate)	59.6	27.2	75.8	84.4	1.23×
No reliability scoring	59.6	27.2	75.8	84.4	2.67×
No attribute binding	57.6	26.0	75.5	83.9	2.67×

SAY — Harsh: On CHAIR, the full toolkit lowers the share of captions with at least one hallucinated object from 63.8 to 57.2 percent — 6.6 points, about 10% relative — and hallucinated object mentions from 29.6 to 25.5 percent, about 14% relative. Object recall falls by only 0.8 points and captions lose 2.5 words on average, so we are not getting this by deleting lots of content. The price is latency: 2.67 times greedy decoding.



Figure 11 (paper Fig. 2). $CHAIR_S$ against latency for the five variants; lower-left is better.



Where the gain comes from: n = 1 (correction only) versus the full toolkit (correction + re-ranking over 3).

Figure 12. Decomposition of the CHAIR gain.

4.7 What the ablations tell us (60 s) — Section VI-C

SAY — Harsh: The ablations answer “which part is doing the work?”. Correcting a single candidate already lowers $CHAIR_S$ by 4.2 points and $CHAIR_I$ by 2.4, at only 1.23 times the latency — so most of the gain comes from claim-level correction itself. Adding two sampled candidates with reliability-guided re-ranking gives a further 2.4 and 1.7 points, at an extra 1.44 times latency. Turning off reliability scoring reproduces the n = 1 numbers exactly: without a score, candidate 1 is always kept, so the extra candidates are generated and paid for but never used. Re-ranking is the only route through which they help. Removing the attribute probe changes $CHAIR_S$ by just two captions out of 500 — which we expect, because CHAIR has objects but no attribute ground truth. So we're honest that attribute correction is not yet measured by these benchmarks.

4.8 POPE results (45 s) — Table IV

Method	Split	Acc.	Prec.	Rec.	F1	Yes %	Abst. %
Base	Random	87.6	96.9	77.6	86.2	40.0	0.0
Toolkit	Random	87.7	**97.8**	77.2	86.3	39.4	0.4
Base	Popular	86.6	94.7	77.6	85.3	41.0	0.0
Toolkit	Popular	86.6	**95.1**	77.2	85.2	40.6	0.5
Base	Adversarial	84.3	89.6	77.6	83.2	43.3	0.0
Toolkit	Adversarial	84.4	**90.3**	77.2	83.2	42.7	0.5

SAY — Harsh: On POPE the effect is small, and we say so plainly. Accuracy moves by at most 0.1 points; precision rises on every split, by up to 0.9 points. The reason is mechanical: the toolkit can only turn a “yes” into a “no”, and LLaVA already says yes to only 40 to 43 percent of a benchmark designed to be balanced. Its dominant error is missing objects — recall is 77.6 — and verifying the claims it does make cannot fix a claim it never made. We remove a few false positives, about 11 to 16 answers per split, at a small cost in true positives.

4.9 Calibration (20 s)

SAY — Harsh: We also measured how well R predicts correctness on POPE: the expected calibration error is between about 0.21 and 0.27. So R is useful for ranking candidates, but it is not yet a calibrated confidence — that needs human-annotated validation.

4.10 The audit log (30 s)

DO

1. Open the **Experiments** page. Click today's Scene B run to reopen it.

SAY — Harsh: Every live analysis, including the three you just watched, is stored in SQLite with the image hash, all candidates, every claim score, the evidence, the actions taken and the decision. So any result can be reopened, compared later, and used for regression testing.

4.11 Limitations and future work (60 s) — Sections VII and VIII

Limitation (paper Section VII)	What we plan
Bounded by the verifier: small or occluded objects can be missed; rare categories fall outside the vocabulary	Stronger detectors; report recall alongside every result
Rule-based extraction; closed vocabulary of 11 colours and 8 materials	Richer vocabulary and relation/count claims
Colour is sensitive to lighting and boundaries	Per-type calibration of the attribute score
Attribute correction is not measured by POPE or CHAIR	Winoground, SugarCrepe and a human-audited binding-error rate
One generator, one seed, one run; no confidence intervals	Paired resampling, significance tests, more generators
Object thresholds tuned on POPE and reused for captions	Tune per benchmark on held-out captions
R is not a calibrated confidence	Human-annotated validation
2.67× latency	n = 1 gives most of the gain at 1.23×; reduce verification cost

SAY — Harsh: To be clear about the limits: we are only as good as our detector; our vocabulary is closed; colour depends on lighting; attribute correction isn't yet measured by these benchmarks; and these are results from one model, one seed and one run. Next we'll evaluate attribute binding directly on Winoground, SugarCrepe and a human-audited error rate, add significance tests and more generators, and cut the latency.

4.12 Close (20 s)

SAY — Harsh: To sum up: a training-free, model-agnostic layer that verifies each claim against the image, binds attributes to the right object, and corrects only what the evidence contradicts. On LLaVA-1.5-7B it cuts $CHAIR_S$ by 6.6 points and $CHAIR_I$ by 4.1 with less than a point of recall loss. Thank you — we're happy to take questions.

Questions you own

If asked…	Answer
Why not abstain whenever R is low?	Because evidence-backed corrections are still valuable; Scene B has R = 0.40 and is correctly fixed.
Do extra candidates help on their own?	Only through re-ranking. With scoring disabled, candidate 1 is always kept, so the extras cost latency and change nothing (see the ablations).
Is a 6.6-point drop significant?	Single run, single seed, 500 captions. It needs paired resampling and more generators — it's listed as a limitation.
Why is the full pipeline 2.67× slower?	Three generations plus detection and CLIP per claim. n = 1 gives most of the gain at 1.23×.
Can someone reproduce this without a GPU?	Yes. Download the generations and cache from the Hugging Face dataset and run process --cache-only; it re-scores in about 20 s.
Why does POPE barely move?	The toolkit only converts yes → no, and LLaVA's main POPE error is missed objects (recall 77.6%).

Appendix A — Run sheet and timing

Clock	Who	Section	On screen
0:00	Yash	The problem, related work, formulation	Problem figure, comparison table
3:30	Yash	Contributions; how hallucination is measured	Script pages
5:00	Prithvi	Architecture, candidate generation, claim extraction	system-overview.png, analysis-pipeline.png
9:30	Prithvi	Scene A (live Gemini)	Analyze → Claims
11:00	Kavyansh	Verification, counts, relations	Verification figure
13:00	Kavyansh	Scene B; correction operations; terminal swap	Analyze → Evidence, claims, diff; terminal
16:00	Kavyansh	Reliability score; thresholds and weights	Reliability figure
18:00	Harsh	Claim and answer policy; Scene C (re-rank)	Policy figures; Analyze → Decision, Candidates
21:00	Harsh	Setup, pipeline, CHAIR, ablations, POPE, calibration	Evaluation page; benchmark figures
24:00	Harsh	Audit log, limitations, close	Experiments page
25:00	All	Questions	—

Appendix B — Before the demo

- Start everything: powershell -ExecutionPolicy Bypass -File scripts\dev.ps1. Check http://127.0.0.1:8010/health returns ok and http://127.0.0.1:3000/analyze loads.
- Confirm **results/tuned.yaml** exists (object thresholds 0.15 / 0.20). It is gitignored, so it is not in a fresh clone.
- Run scenes A, B and C once so B and C return from the cache in seconds. Keep both override captions in a notes file.
- Run .venv\Scripts\python demo_paper_example.py once in the terminal you will use.
- Open the Evaluation page and check the POPE and CHAIR tables load. Close other heavy apps: verification runs on the CPU.

Appendix C — If something breaks

Problem	What to do
Gemini error or no Wi-Fi	Prithvi reopens this morning's Scene A run from the Experiments page. Scenes B and C need no API.
The dashboard shows ECONNREFUSED	The backend is down. Re-run scripts\dev.ps1, wait 10 s, refresh.
An analysis takes minutes	The image is not cached (CPU verification). Use the pre-run images, which return in seconds.
Numbers differ from this script	Check results/tuned.yaml is present; without it the object thresholds are 0.30 / 0.45.
Evaluation page empty	results/benchmark_report.json is missing; copy it back from the Hugging Face dataset.

Appendix D — Paper versus implementation (checked 30 Oct 2026)

Matches the code

- Generation: n = 3, temperature 0.7, seed 42, 128 tokens; greedy candidate 1; GENERATION_FAILED keeps its slot.
- Vocabulary: 11 colours and 8 materials as listed; NMS IoU 0.5; the near / next-to rule; the four edits, including recount.
- Eq. 3–5, the floor 0.10, the grounding threshold 0.30, P ≥ 0.5, λ = (0.6, 0.2, 0.2), τ_R = 0.5, and the re-rank and abstain rules in core/policy.py.
- Every number in the abstract and Tables IV and V matches results/RESULTS.md. The Section IV-E worked example (0.867) is reproduced live by Scene C, and Table VI by demo_paper_example.py.

Fixed while preparing this script

- The dashboard folder had no results/tuned.yaml, so live runs used object thresholds 0.30 / 0.45 instead of the paper's tuned 0.15 / 0.20. It was copied in, and all scenes were re-run with it.

Worth fixing in the paper text

- Section IV-A calls the dashboard a module “for human review”; the Review page was removed. Suggest “for inspection and audit”.
- Table I caption: “Claim Types Handled by the Extract” → “…by the Extractor”.
- Section VII: “(vii) Object thresholds are tuned on POPE…” is merged into the previous bullet; give it its own.
- Abstract: “reranking” is split as “re-ranking”; equation numbers sit against the formulas (“d_b(3)”) — add spacing.
- Section V-D says 78 tests: true on the new-ui branch; 81 after merging fix/reliability-weights.
- Section V-D “about 3× faster, scores within ±0.03” for 480 × 800 input: confirm from your measurement notes.

Small code inconsistency

- core/policy.py still says “escalated for human review” when no candidate is reliable. No demo scene hits this branch, so it will not appear on screen.

Appendix E — Glossary

Term	Meaning
Claim	One atomic statement extracted from an answer: an object, attribute, count or relation.
v_i	A claim's verification score in [0, 1] (s_{obj}, s_{attr}, or the count/relation score).
d_b	Grounding DINO's confidence for box b.
p_j	Token confidence of candidate j (white-box models only).
A_j	Agreement: share of candidate j's claims also made by another candidate.
R_j	Reliability of candidate j (Eq. 5).
τ_{lo}, τ_{hi}, τ_R	Correct-below, accept-above, and re-rank thresholds.
$CHAIR_S$ / $CHAIR_I$	Share of captions with a hallucination / share of object mentions that are hallucinated.
ECE	Expected calibration error: how far a score is from the observed accuracy.